2026 청년도약 인재양성 부트캠프

★ 기업 현장에서 배우며, 실무형 인재로 성장 ★

★ 실전인재형 ★ 점프인재형 ★ 실전인재형 ★ 점프인재형 ★ 실전인재형

인공지능 부트캠프

#지능형 프롬프트 엔지니어링
- 인공지능 활용 소프트웨어 개발과 산출물

컴퓨터공학과

김 원 일

인공지능을 활용한 표준 개발 흐름

- 개발 단계별 설정
  - 고정되어 있는 형식이 아니므로 본인에게 맞는 개발 흐름을 찾을 것
  - 각 단계마다 다른 인공지능을 사용하기 때문에 반드시 문서화할 것
  - 문서화된 내용을 다음 단계에 전달해야 이전 단계의 내용이 유지됨
    - 지속적인 인공지능 교체가 이루어진다면 단계별 문서화는 필수
    - 특정 단계 내용 변경이 필요하다면 이전 정보를 먼저 전달하고, 변경 내용을 생성하도록 지시
    - 테스트 단계에서 요구사항의 문제가 확인되면, 요구사항 점검 후 적용할 때도 유용
  - 단계 별 정보 보관과 유지가 개발자에게 필요함
    - 개발자가 직접 수행하지 않아도 지시는 개발자가 직접해야 함
  - 설계 전에 가장 중요한 단계는 요구사항 단계
    - 반드시 요구사항에 대해 정의하고, 설계 단계를 진행해야 함



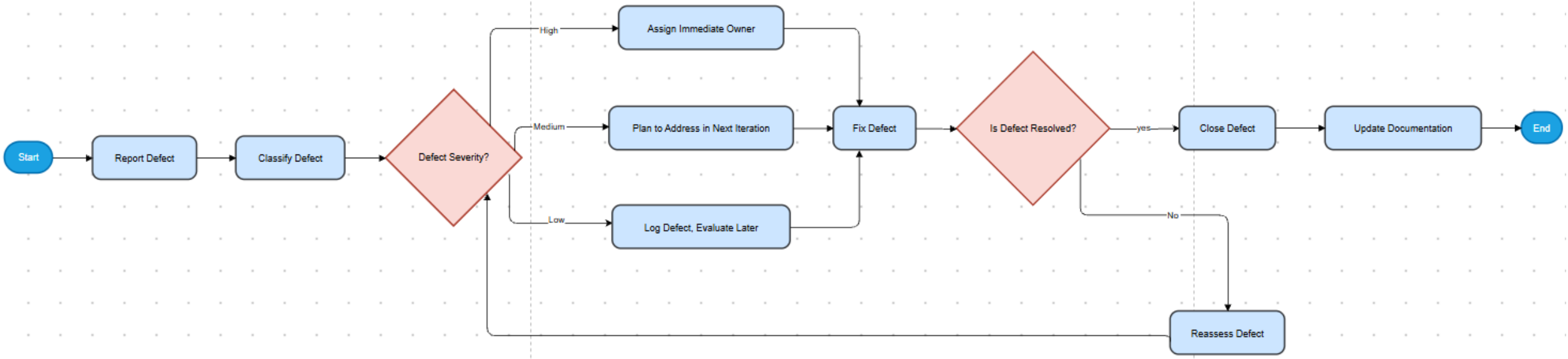
단계별 진행과 산출물

- 근본적인 문제 파악을 통한 목적과 범위를 정의하는 단계
- 개발자
  - 담당자 미팅을 통한 프로젝트의 목표와 성공 지표(KPI) 설정
  - 제약 사항(예산, 일정, 리소스 등) 파악 및 범위 설정
- 산출물
  - 프로젝트 제안과 문제 정의서 및 프로젝트 범위 정의서
- 연관 UML 다이어그램
  - 활동(Activity) 다이어그램으로 비즈니스 흐름과 문제점의 시각화



단계별 진행과 산출물

- KPI(Key Performance Indicator)
  - 핵심 성과 지표라고 하며, 목표 달성을 객관적으로 측정하기 위한 평가 기준
  - IT 분야의 KPI 예제
    - 시스템이 중단 없이 서비스가 제공된 시간의 비율
    - 시스템 오류나 버그 접수 후, 해당 버그 수정에 소요되는 평균 시간
- Activity Diagram 예제



단계별 진행과 산출물

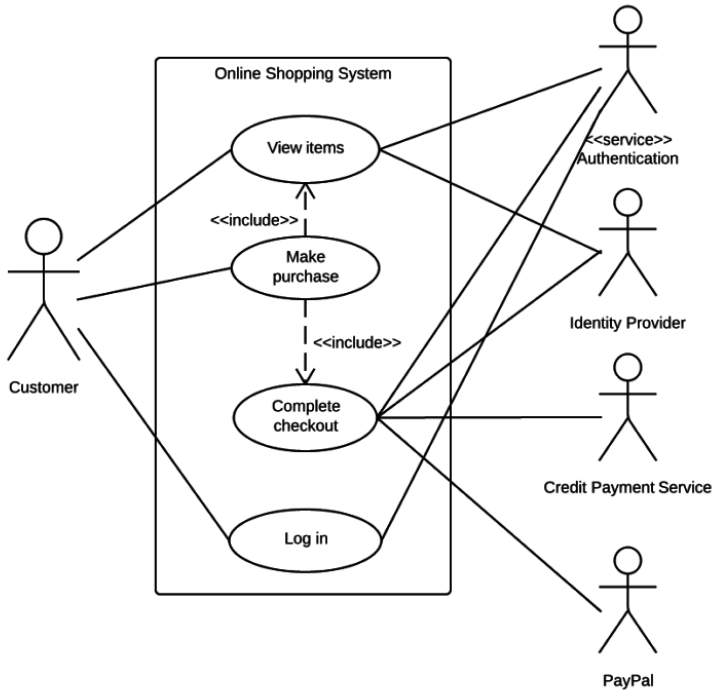
- 문제 해결을 위해 기능/비기능 요구사항을 구체화하는 단계
- 개발자
  - 사용자 스토리 작성 및 MoSCoW를 통한 우선순위 설정
  - 기능/비기능 요구사항에 대한 분석을 통한 요구사항 정의서
- 산출물
  - 소프트웨어 요구사항 명세서(SRS – Software Requirements Specification)
  - 용어 사전(Glossary) 및 사용자 스토리 관리 문서
- 연관 UML 다이어그램
  - Use Case 다이어그램으로 사용자와 시스템의 상호작용 및 기능 범위 정의



단계별 진행과 산출물

- MoSCoW
  - MUST HAVE(Mo) : 반드시! 필수적으로 있어야만 하는 기능
  - SHOULD HAVE(S) : 중요하긴 하지만 필수적이지는 않은 기능
  - COULD HAVE(Co) : 있다면 괜찮을 것 같은 기능
  - WON'T HAVE(W) : 가치가 거의 없거나 전혀 없는 기능

- Use-Case 다이어그램
  - 사용자 관점에서 시스템이 제공하는 기능을 정의
  - 외부 요소 간의 상용 작용을 그림으로 표현한 것
  - 요구사항에 대한 범위를 시각적으로 표현
  - 개발자가 아닌 사람도 쉽게 접근이 가능
  - 무엇(What)에만 집중된 내용으로 구성



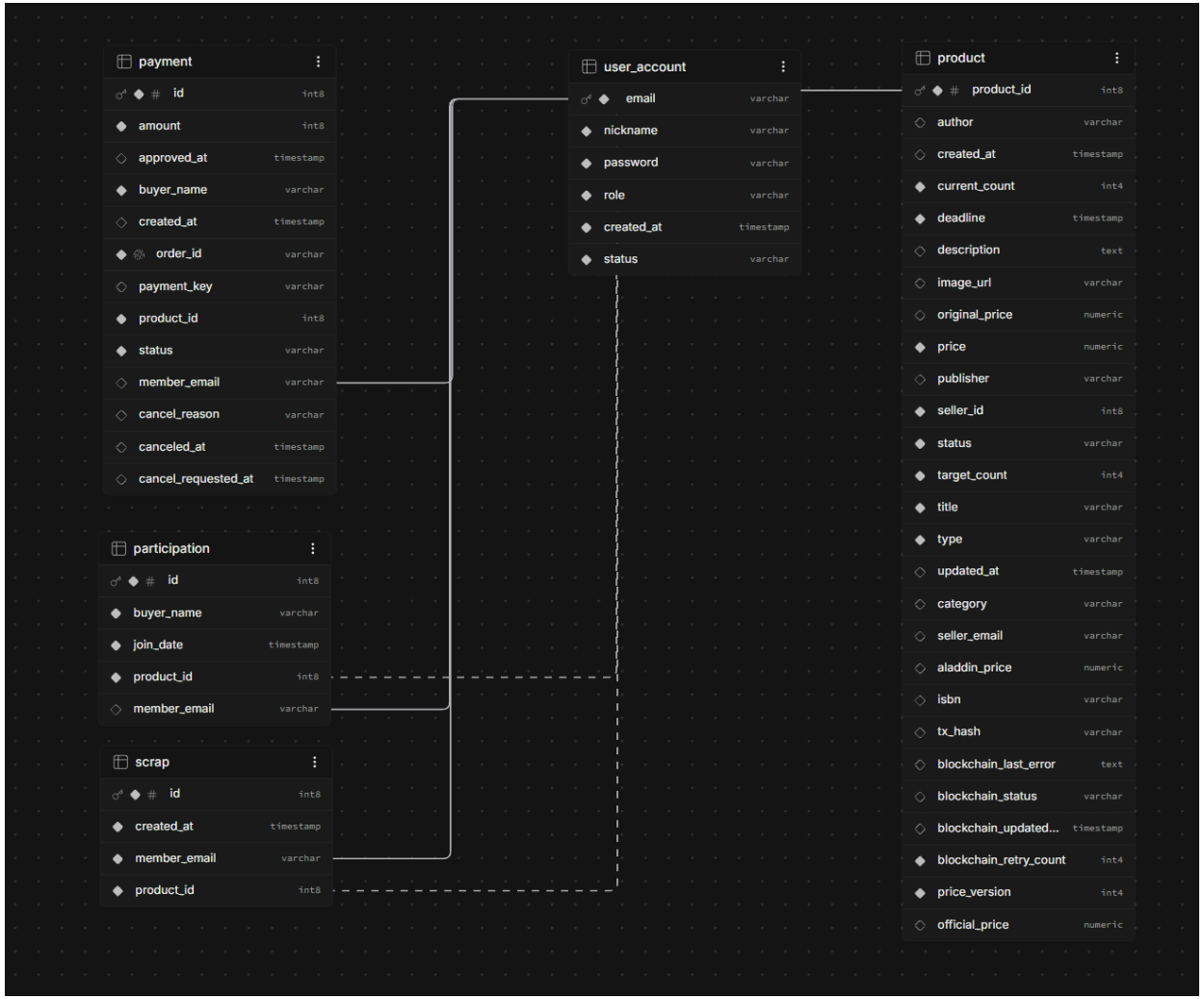
단계별 진행과 산출물

- 요구사항을 바탕으로 데이터 구조, 화면, 인터페이스 등의 설계 진행
- 개발자
  - UI/UX 검토 및 화면 별 데이터 흐름 정의
  - 데이터베이스 모델링(스키마, 컬럼, 인덱스 등의 설계)
  - 프론트 엔드와 백엔드 및 외부 시스템 연동을 위한 API 정의
- 산출물
  - ERD(Entity Relationship Diagram), API 명세서 초안과 화면/인터페이스 설계서
- 연관 UML 다이어그램
  - Sequence 다이어그램으로 객체 간 메시지 교환 순서 정의
  - Class 다이어그램으로 객체들의 속성, 메서드 및 관계 정의



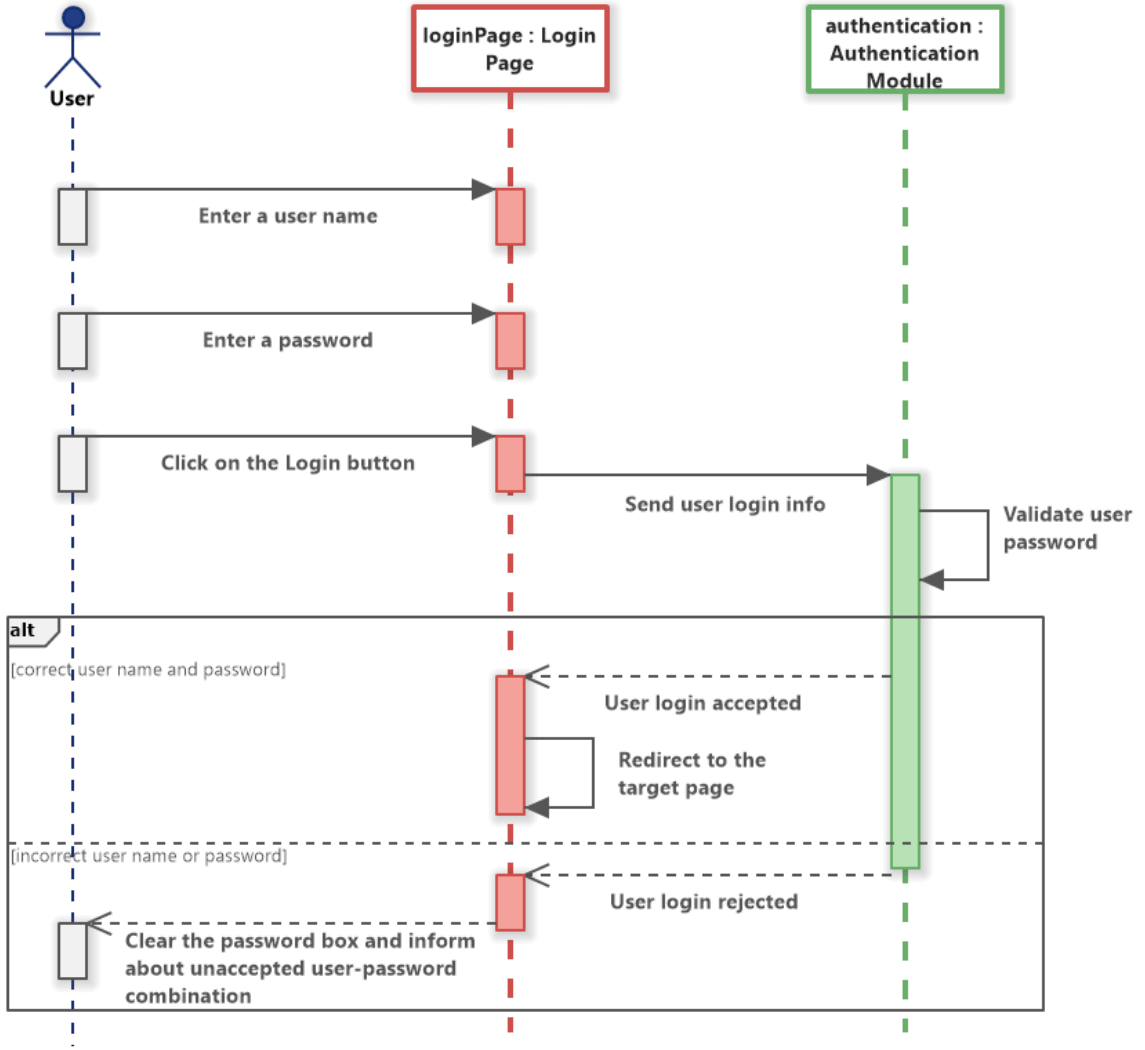
단계별 진행과 산출물

- ERD(Entity-Relationship Diagram)
  - 데이터베이스의 정보 관리 상태를 표현한 다이어그램
  - Entity 간의 관계를 이용하여 정보의 연결 관계를 시각화한 설계도



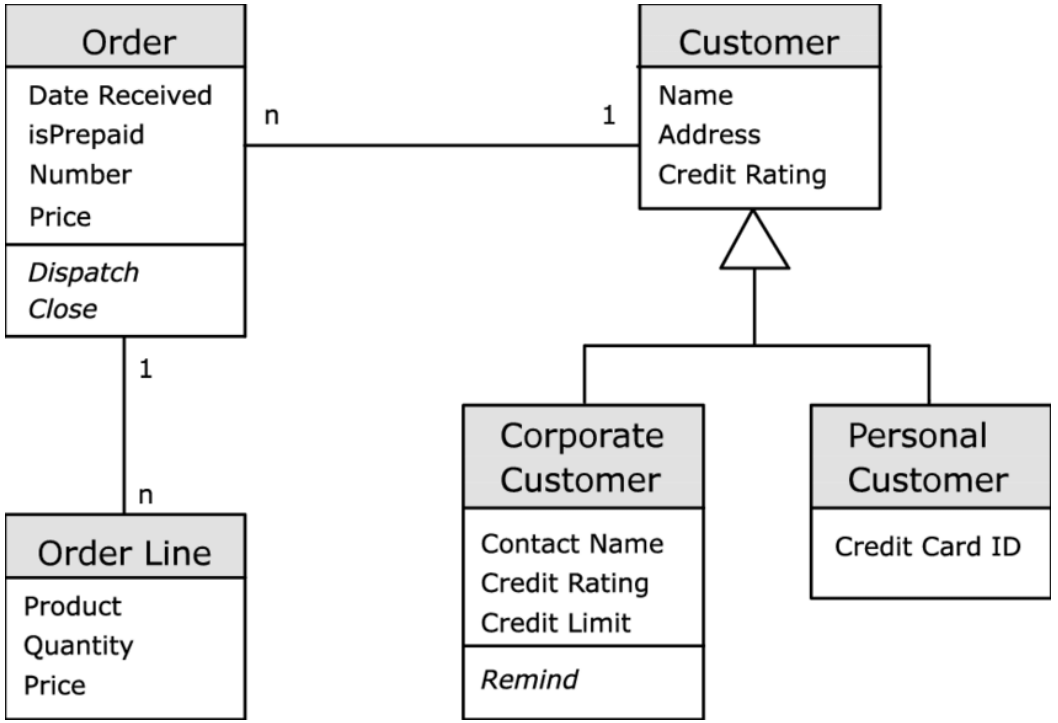
단계별 진행과 산출물

- Sequence Diagram
  - 시간 흐름에 따라 시스템 객체가 어떤 정보를 주고 받으며 상호작용을 하는지를 명세한 다이어그램
  - 특정 기능마다의 시나리오 구축용
  - 흐름 확인과 함수 호출 및 값의 흐름과 기능 수행 후에 반환되는 값에 대한 이해를 직관적으로 확인할 수 있음
  - 이를 바탕으로 클래스 다이어그램을 작성하는데 사용 가능



단계별 진행과 산출물

- Class Diagram
  - 클래스의 정적 구조를 시각화하여 표현한 다이어그램
  - 클래스의 속성과 메서드 및 클래스 간의 관계를 표현
  - 클래스 다이어그램을 이용한 코드 생성이 가능



단계별 진행과 산출물

- 소프트웨어의 전체 뼈대와 기술 스택 및 인프라 및 배포 환경 결정
- 개발자
  - 전체 시스템 아키텍처 결정과 기술 스택 선정 및 검증(PoC)
  - 인프라 및 네트워크 구성과 CI/CD 파이프라인 및 보안 정책 설계
  - 코딩 규칙과 git 브렌치 전략 등을 수립
- 산출물
  - 소프트웨어 아키텍처 정의서와 인프라 구성도, 코딩 규칙과 협업 가이드
- 연관 UML 다이어그램
  - Component 다이어그램으로 시스템의 물리/논리적 모듈과 의존성 정의
  - Deployment 다이어그램으로 소프트웨어의 전체 배치도 작성



단계별 진행과 산출물

- PoC(Proof of Concept)
  - 개념 증명을 뜻하며, 새로운 아이디어, 기술 등이 실제 구현 가능한지 확인
  - 디자인이나 완성도와 관계 없이 기술적으로 가능한지 여부만 확인
- Prototype
  - 어떤 형태로 작동하는지에 대한 흐름과 UI 시각화
  - 아이디어를 구체적인 디자인과 화면 흐름으로 만들어 공유
- MVP(Minimum Viable Product)
  - 최소 기능 제품으로 수요 확인을 위한 용도로 사용
  - 핵심 기능만 만들어 실제 사용자에게 출시하여 반응을 확인하는 용도
- Pilot Test
  - 실제 운영 환경에서도 문제없이 동작하는지를 점검
  - 정식 출시전 제한된 환경이나 일부 사용자에게 운영 안정성 평가가 목적

단계별 진행과 산출물

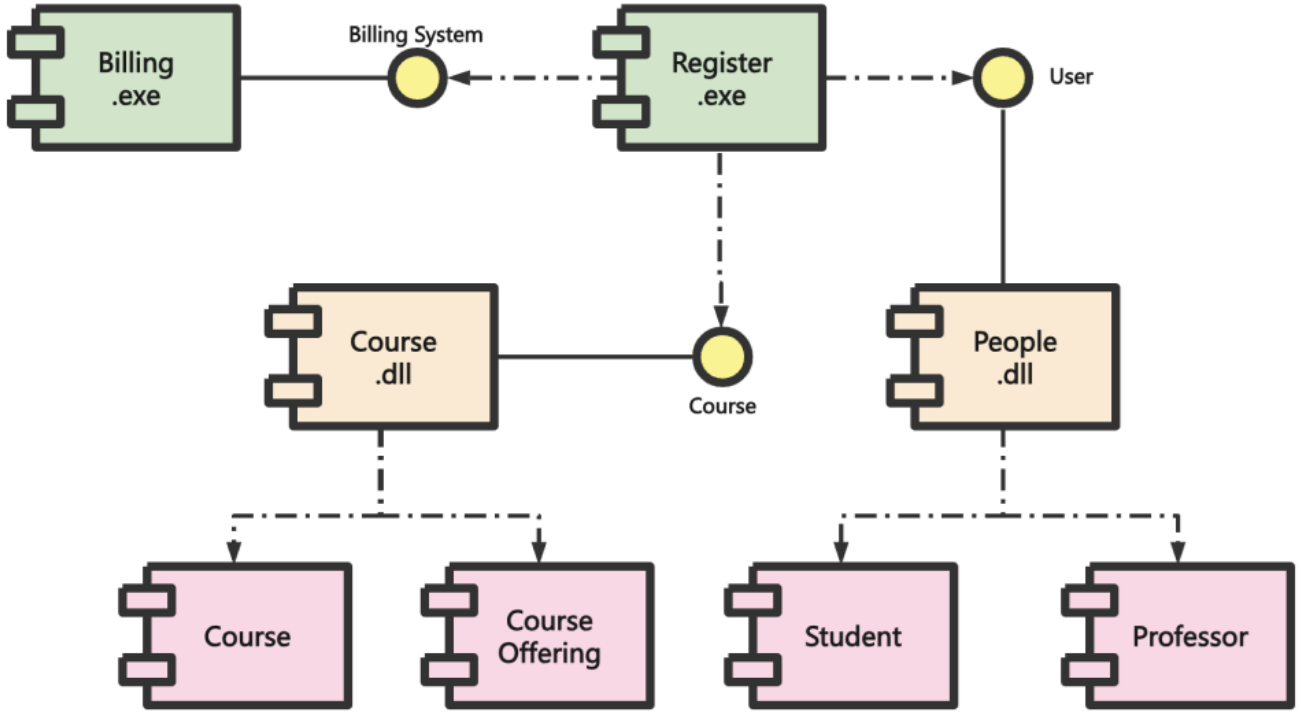
- CI(Continuous Integration)/CD(Continuous Deployment)
  - 직속적 통합은 코드 변경 사항을 저장소에 자동으로 자주 통합하여 관리
  - 지속적 배포는 코드 변경 사항 통합, 테스트 후에 배포까지의 단계를 총망라
  - 현재 수행하고 있는 프로젝트에 CI/CD가 포함되어 있는지 확인
  - 단순히 git을 이용한 소스 코드 관리는 CD/CD와 전혀 관계가 없음
  - 가장 많이 사용하는 CI/CD 도구는 Jenkins

    ➔ 가장 유명하며, 단순 CI부터 완전한 CD까지 모든 것을 설계하고 실행 가능
  - 아래는 Red Hat의 CI/CD 설명 다이어그램
    - https://www.redhat.com/ko/topics/devops/what-is-ci-cd



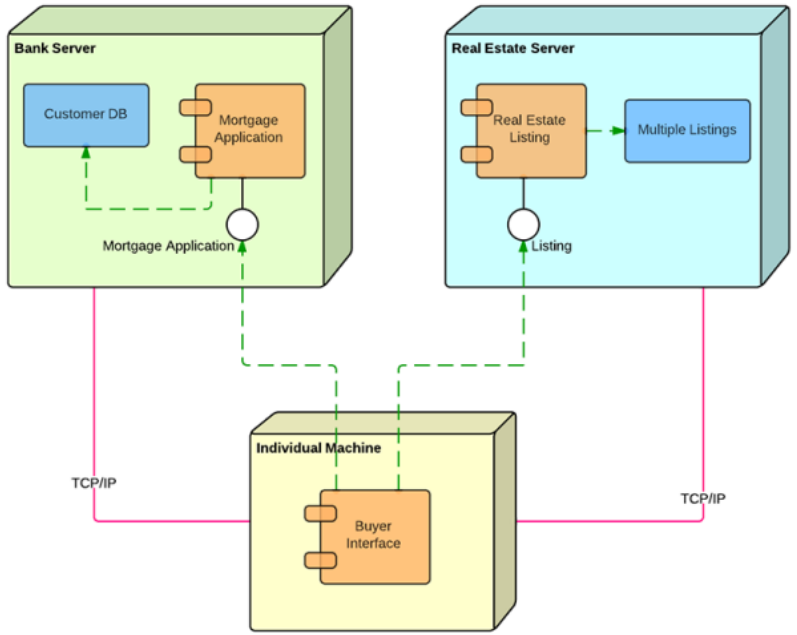
단계별 진행과 산출물

- Component 다이어그램
  - 소프트웨어 시스템의 물리적 구조와 구성 요소의 종속성을 표현
  - 실행파일과 라이브러리가 실행 시에 어떻게 동작하는지를 표현
  - 컴포넌트 형태로 설계하면 각 구성요소가 독립적으로 동작
  - 독립적이기 때문에 언제든지 교체나 업데이트가 가능



단계별 진행과 산출물

- Deployment 다이어그램
  - 소프트웨어가 물리적 하드웨어 및 실행 환경에 배치되는지를 시각적으로 표현
  - 물리적 시각화로 서버, 클라이언트, DB와 소프트웨어와의 관계 표현
  - 시스템의 성능, 확장성, 가용성, 보안, 유지보수성을 설계하고 논의 및 검토
  - 소프트웨어 배포 환경 등을 명확하게 정의



단계별 진행과 산출물

- 설계 산출물들을 이용해 실제 코드 작성과 형상 관리 및 빌드 환경 구축
- 개발자
  - 보일러플레이트 코드 작성과 비즈니스 로직 구현
  - 코드 규칙을 준수한 코드 생성과 단위 테스트 코드 작성
  - git을 활용한 형상 관리 및 코드 리뷰를 수행하면서 구현 단계를 진행
  - 빌드 및 배포 등을 통해 기초 동작 검증
- 산출물
  - 소스 및 단위 테스트 코드와 API 상세 명세서 및 빌드/배포 자동화 스크립트
- 연관 UML 다이어그램
  - State 다이어그램으로 상태 변화 추적을 수행

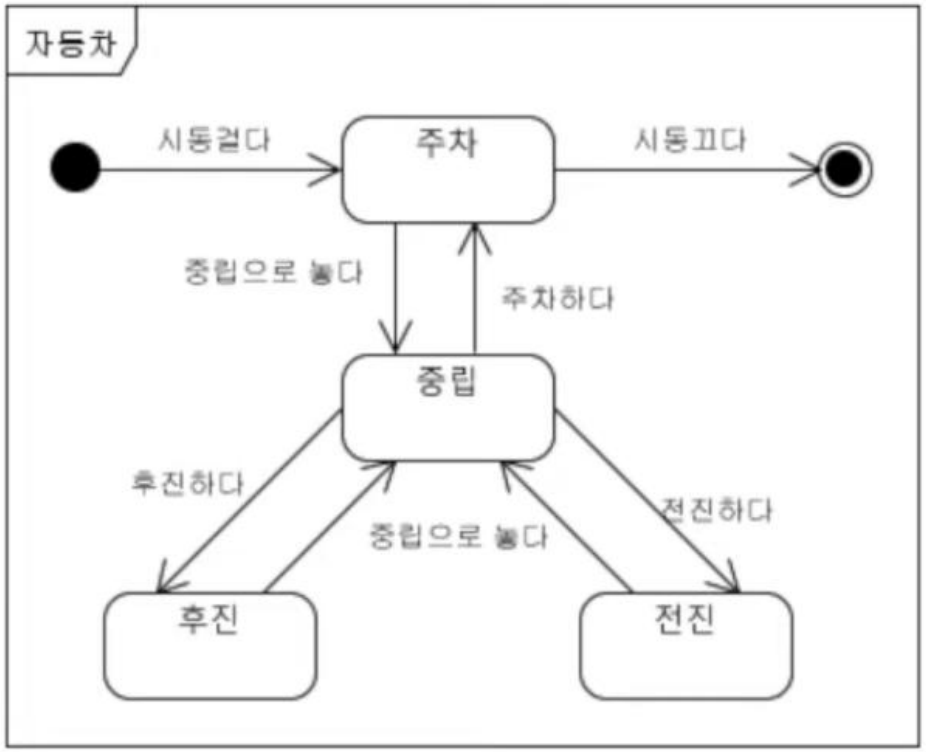


단계별 진행과 산출물

- 보일러플레이트 코드(Boilerplate Code)
  - 최소한의 수정으로 여러 곳에서 반복해서 재사용되는 표준화된 코드
  - 프로젝트 시작 시, 기본 구조나 설정을 새로 작성하지 않고 즉시 활용 가능
  - 팀 단위 개발이나 여러 모듈에서 동일한 구조로 사용
  - 표준화를 위해 팀이나 회사 단위로 작성되므로 안정적으로 사용 가능
  - 대표적인 예시 : HTML의 <html>, <body>, <head>와 같은 태그
- 코딩 규칙
  - 함수는 로직이 복잡하여 길게 작성해야 하는 경우가 아니면 20라인 이내로 작성
  - 함수 이름은 Snake-Case를, 변수 이름은 Camel-Case로 작성
  - 모든 변수는 선언과 동시에 반드시 0으로 초기화해야 함
  - 변수 선언은 한 줄에 반드시 하나의 변수만 선언해야 함
  - 코드를 작성하는데 필요한 규칙을 설정하고, 해당 규칙대로만 작성

단계별 진행과 산출물

- State Diagram
  - 객체나 시스템이 시간의 흐름에 따른 상태 변화를 시각적으로 표현
  - 특정 이벤트가 발생하면, 다른 상태로 전이되는지를 확인할 수 있음
  - 상태 변화가 많고, 뚜렷한 시스템에 필수



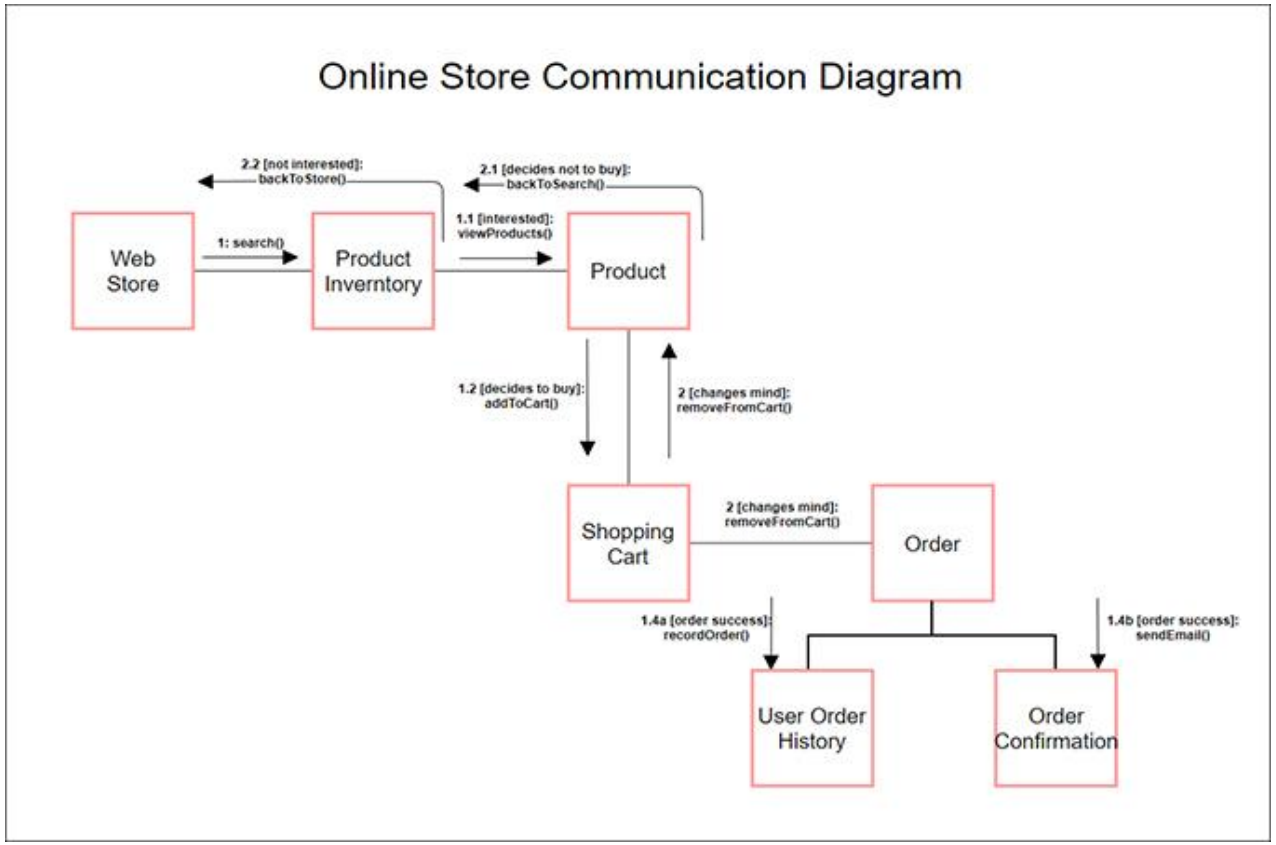
단계별 진행과 산출물

- 구현 소프트웨어의 결함과 취약점 등의 다각적인 검증
- 개발자
  - 통합 및 시스템 테스트, 결함 및 취약점 검증으로 문제의 원인 분석과 수정
  - 회귀(Regression) 테스트 자동화 환경 구축
- 산출물
  - 테스트 케이스 시트(TC), 결함/버그 리포트
  - 테스트 결과 보고서(커버리지, 성능 지표 포함)
- 연관 UML 다이어그램
  - Communication 다이어그램으로 동시성 이슈와 흐름 점검



단계별 진행과 산출물

- Communication 다이어그램
  - 객체들이 서로 주고받는 메시지와 객체들 간의 연결 관계를 시각화
  - 객체 간의 협업 구조 파악에 용이하고, 실제 런타임에서 검증용으로 사용



단계별 진행과 산출물

- 테스트 결과의 이슈 분석과 수정
- 개발자
  - 배포와 모니터링 및 로그 수집, 각종 산출 문서들의 현행화
  - 기술적 문제점과 향후 개선을 위한 백로그 작성
  - 실제로 발생할 수 있는 장애 상황에 대비한 런북(Runbook) 작성
- 산출물
  - 운영자 매뉴얼 및 장애 대응 가이드, 현행화 된 아키텍처 및 API 문서
- 연관 UML 다이어그램
  - Package 다이어그램으로 패키지 구조 및 모듈 간 의존성 요약

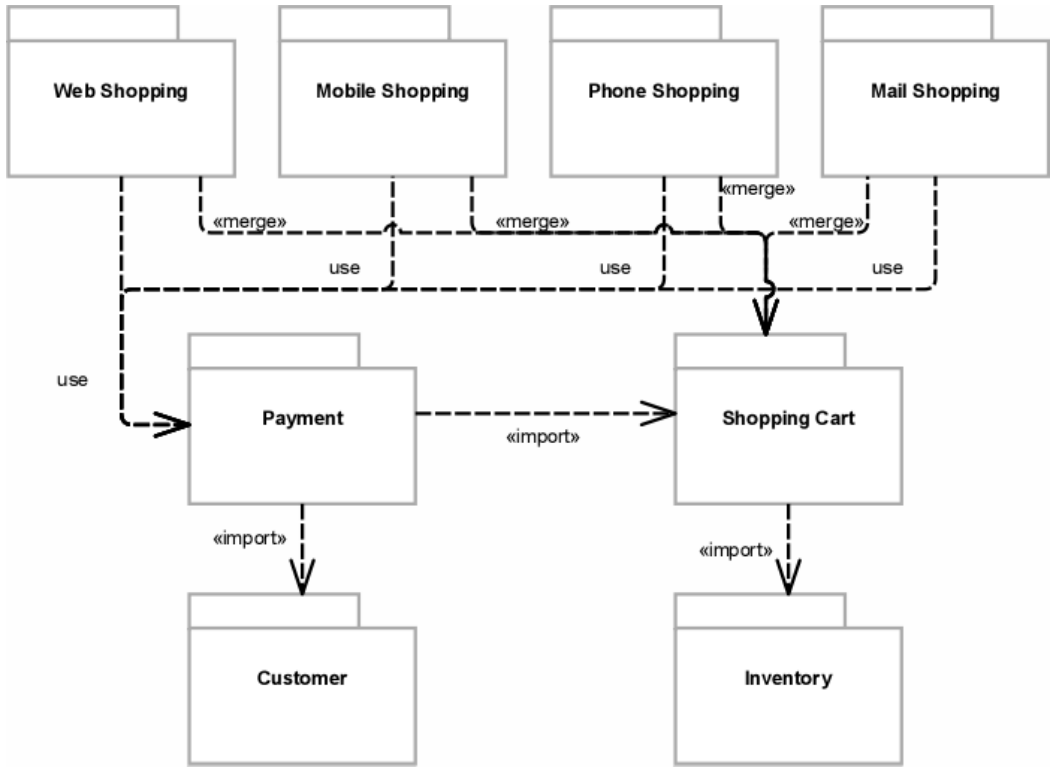


단계별 진행과 산출물

- Runbook
  - 시스템 운영, 장애 대응, 정기 작업 등을 원활하게 수행하는데 필요한 매뉴얼
  - 문제 발생에서 무엇을 확인하고, 어떤 순서로 대응해야 하는지를 정리한 문서
  - 문서 표준화를 통해 누구나 동일하게 문제 해결할 수 있도록 작성
  - 숙련자나 개발자만 알고 있는 노하우를 공개하여 공유
  - 장애 발생 시 원인 파악 및 조치 시간을 최소화
  - 구성 요소
    - 대상 및 실시 조건 : 어떤 시스템이나 환경을 대상으로 하는가
    - 사전 확인 및 준비물 : 작업 시작 전에 점검 사항과 권한
    - 구체적 작업 절차 : 단계별 실행 및 확인 순서 (명령어나 자동화 스크립트)
    - 판단 및 분기 조건 : 예상 밖의 결과가 나온 경우의 판단과 대처 방법
    - 이상 시 대응 : 문제가 해결되지 않은 경우의 보고 대상과 전파 방법
    - 완료 확인 : 작업이 제대로 종료 되었는지 검증하는 방법

단계별 진행과 산출물

- Package 다이어그램
  - 시스템의 요소들을 그룹화(패키징)하고, 의존 관계를 시각화
  - 소프트웨어의 클래스, 인터페이스, 컴포넌트 등을 추상화하여 표시
  - 모듈 간 결합도와 상위 수준의 구조적 의존 관계 파악에 유용



설계 평가 문제

로또 번호 생성기 프로그램을 작성하려고 한다. 언어와 플랫폼 모두 상관 없이 로또 번호를 생성하고, 이전에 생성된 번호 목록을 볼 수 있으면서 시각적으로 확인이 쉬운 형태의 로또 번호 생성 프로그램 문제 정의와 요구사항을 문서로 만들어보자.



위 프로그램의 설계를 진행할 때 아래 내용을 포함한 문서를 생성하여 e-class에 업로드
- 프로젝트 개요, 문제 정의, 주요 사용자 및 이해관계자, 기능 요구사항, 비기능 요구사항, 사용자 스토리, 우선순위, 제외 범위 ➔ 지난 시간에 작성한 문서 형태(project.md)
- 디렉터리와 파일 구조 문서(files.md), 아키텍처 설계 문서(arch.md)

* 위의 md 파일 형식의 산출 문서들을 본인 이름으로 압축하여 제출